

2.4.1 Sorting Algorithms: Selection Sort

سلیکشن سارٹ (Selection Sort) میں فہرست کے غیر ترتیب شدہ حصے سے سب سے چھوٹا عدد چن کر اسے سامنے اس کی درست جگہ پر رکھا جاتا ہے۔

CS XI

Chapter 2: Computational Thinking & Algorithm

Lecture 07 of 11

★ Golden

2.4.1 Sorting Algorithms: Selection Sort ★ Golden

LEARN IT

Selection Sort is a simple sorting algorithm that works by repeatedly **selecting the smallest element** from the unsorted portion of the list and moving it to its correct position at the front.

Think of it as filling seats from the front: first find who belongs in position 1, then position 2, and so on. In the textbook version, the element at position K is compared with every element after it; whenever a smaller one is found, the two are swapped. So at the end of pass K, the K-th smallest value sits in position K.



Algorithm (textbook)

```

Step 1: Start
Step 2: Set DATA := {8,4,1,9,3}
Step 3: Set N := 5
Step 4: Repeat Step 5 and 6 for
        K = 1 to N
Step 5: Set PTR := K+1
Step 6: Repeat while PTR ≤ N:
    a: If DATA[K] > DATA[PTR]:
        1. Set temp = DATA[K]
        2. Set DATA[K] = DATA[PTR]
        3. Set DATA[PTR] = temp
    b: Set PTR := PTR + 1
Step 7: End
  
```

Pseudocode

```

Start
DATA = {8,4,1,9,3}
N = 5
For outer = 0 to N-1
    For inner = outer+1 to N-1
        IF DATA[outer] > DATA[inner]
            Temp = DATA[outer]
            DATA[outer] = DATA[inner]
            DATA[inner] = Temp
        End If
    End For
End For
End
  
```

REMEMBER IT

Teacher's tip — spot the difference: in Selection Sort the inner loop starts at `outer + 1` and compares with the *fixed* position `DATA[outer]`. In Bubble Sort the inner loop always starts at `0` and compares *neighbours* `DATA[inner]` and `DATA[inner+1]`. Bubble pushes the largest to the **end**; Selection brings the smallest to the **front**.

WORKED EXAMPLE

Worked example: sort 8, 4, 1, 9, 3 in ascending order (textbook method). The two positions in brackets are being compared.

Pass 1 — find the smallest for Position 1

Compare	List	Pos1 > PosX?	Action	List after
Pos1 vs Pos2	[8], [4], 1, 9, 3	8 > 4 Yes	Swap	4, 8, 1, 9, 3
Pos1 vs Pos3	[4], 8, [1], 9, 3	4 > 1 Yes	Swap	1, 8, 4, 9, 3
Pos1 vs Pos4	[1], 8, 4, [9], 3	1 > 9 No	No change	1, 8, 4, 9, 3
Pos1 vs Pos5	[1], 8, 4, 9, [3]	1 > 3 No	No change	1 , 8, 4, 9, 3

1 is now permanently in Position 1.

Pass 2 — find the smallest for Position 2

Compare	List	Pos2 > PosX?	Action	List after
Pos2 vs Pos3	1, [8], [4], 9, 3	8 > 4 Yes	Swap	1, 4, 8, 9, 3
Pos2 vs Pos4	1, [4], 8, [9], 3	4 > 9 No	No change	1, 4, 8, 9, 3
Pos2 vs Pos5	1, [4], 8, 9, [3]	4 > 3 Yes	Swap	1, 3 , 8, 9, 4

Pass 3 — find the smallest for Position 3

Compare	List	Pos3 > PosX?	Action	List after
Pos3 vs Pos4	1, 3, [8], [9], 4	8 > 9 No	No change	1, 3, 8, 9, 4
Pos3 vs Pos5	1, 3, [8], 9, [4]	8 > 4 Yes	Swap	1, 3, 4 , 9, 8

Pass 4 — find the smallest for Position 4

Compare	List	Pos4 > PosX?	Action	List after
Pos4 vs Pos5	1, 3, 4, [9], [8]	9 > 8 Yes	Swap	1, 3, 4, 8, 9

Sorted list: 1, 3, 4, 8, 9. Totals: 4 + 3 + 2 + 1 = **10 comparisons**, **6 swaps** in this textbook method.

Pass	List after pass	Fixed at the front
Start	8, 4, 1, 9, 3	—
1	1, 8, 4, 9, 3	1
2	1, 3, 8, 9, 4	1, 3

3	1, 3, 4, 9, 8	1, 3, 4
4	1, 3, 4, 8, 9	all

WORKED EXAMPLE

Selection Sort in Python (follows the textbook pseudocode):

```
data = [8, 4, 1, 9, 3]
n = len(data)
for outer in range(n - 1):
    for inner in range(outer + 1, n):
        if data[outer] > data[inner]:
            temp = data[outer]
            data[outer] = data[inner]
            data[inner] = temp
    print("After pass", outer + 1, ":", data)
print("Sorted list:", data)
```

OUTPUT

```
After pass 1 : [1, 8, 4, 9, 3]
After pass 2 : [1, 3, 8, 9, 4]
After pass 3 : [1, 3, 4, 9, 8]
After pass 4 : [1, 3, 4, 8, 9]
Sorted list: [1, 3, 4, 8, 9]
```

WORKED EXAMPLE

The "classic" Selection Sort (one swap per pass). Many books first *find the position of the smallest* value in the unsorted part, and swap only once at the end of each pass. The result is the same; only the swaps are fewer.

```
data = [8, 4, 1, 9, 3]
n = len(data)
for outer in range(n - 1):
    smallest = outer
    for inner in range(outer + 1, n):
        if data[inner] < data[smallest]:
            smallest = inner
    data[outer], data[smallest] = data[smallest], data[outer]
    print("After pass", outer + 1, ":", data)
```

OUTPUT

```
After pass 1 : [1, 4, 8, 9, 3]
After pass 2 : [1, 3, 8, 9, 4]
After pass 3 : [1, 3, 4, 9, 8]
After pass 4 : [1, 3, 4, 8, 9]
```

Here only 4 swaps are made (at most $N - 1$). This is why Selection Sort is said to perform fewer swaps than Bubble Sort. In exams, use the textbook method unless the question asks otherwise.

Advantages and disadvantages

Advantages	Disadvantages
Very easy to code.	Time grows very fast with list size (nested loops, about N^2 comparisons).
Performs fewer swaps overall compared to Bubble Sort.	Must finish every single loop — it is "blind" to an already sorted list.
Does not need extra memory.	Not stable (it might change the relative positions of equal items).

WORKED EXAMPLE

Why "not stable"? Take two equal marks, 5_a (Ali) and 5_b (Bilal), in the list $5_a, 5_b, 2$. Pass 1 compares 5_a with 5_b (no swap), then 5_a with $2 \rightarrow \text{swap} \rightarrow 2, 5_b, 5_a$. Now Bilal is before Ali, although Ali came first originally. Bubble Sort never does this, because it swaps only neighbours that are strictly out of order.

COMMON MISTAKE

In the textbook method, the comparison is always with the **fixed position** DATA[K] (Pos1 in pass 1, Pos2 in pass 2), not with the neighbour. Students often mix this up with Bubble Sort and compare neighbours. Also, after a swap, the *new* value at Pos K is used for the next comparison.

اردو میں سمجھیں

سلیکشن سارٹ (Selection Sort) میں فہرست کے غیر ترتیب شدہ حصے سے سب سے چھوٹا عدد چن کر اسے سامنے اس کی درست جگہ پر رکھا جاتا ہے۔ ہر پاس میں ایک مقررہ جگہ (Position) کا موازنہ اس کے بعد والے تمام عناصر سے ہوتا ہے۔ اس کا کوڈ آسان ہے اور ببل سارٹ سے کم تبادلی (Swaps) ہوتے ہیں، لیکن یہ مستحکم (Stable) نہیں اور ترتیب شدہ فہرست پر بھی پورے لوپ چلاتا ہے۔

EXAM TIP

Typical questions: "Sort ... using Selection Sort, showing all steps", "Differentiate between Bubble Sort and Selection Sort" and "Why is Selection Sort not stable?" In the difference question mention: what is compared (neighbours vs fixed position), where the inner loop starts (0 vs outer+1), where items settle (end vs front), stability, and number of swaps.

CHECK YOURSELF

1. In Selection Sort, where does the inner loop start?
2. Using the textbook method, show 7, 2, 5 after pass 1.
3. Which of the two sorts is stable?

Answers: 1. At outer + 1 (PTR = K + 1). 2. Pos1 vs Pos2: 7 > 2 swap → 2, 7, 5; Pos1 vs Pos3: 2 > 5 no → 2, 7, 5. 3. Bubble Sort.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Selection sort each pass:

- a. Selects the minimum from the unsorted part and swaps it into place
- b. Swaps only adjacent
- c. Needs a sorted list first
- d. Uses a stack

MCQ 2. After k passes, the first k items are:

- a. The k smallest in order
- b. Random
- c. The k largest
- d. Deleted

MCQ 3. Comparisons are about:

- a. $O(n^2)$ even if nearly sorted
- b. $O(n)$
- c. $O(1)$
- d. $O(\log n)$

MCQ 4. Swaps per pass: at most:

- a. One
- b. n
- c. n^2
- d. Zero always

MCQ 5. Selection vs bubble: selection usually has:

- a. Fewer swaps
- b. Fewer comparisons
- c. Better Big O
- d. A hash table

MCQ 6. On 8,4,1 min of whole list is:

- a. 1, swapped to front
- b. 8
- c. 4
- d. 9

Extra short questions

1. Define selection sort.
2. One pass on 8,4,1,9,3.
3. Why still $O(n^2)$ if data is sorted?
4. Stable?

Extra long question

1. Full trace of selection sort on 8,4,1,9,3. Count swaps. Compare swap count with bubble on the same list.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Repeatedly select the smallest remaining item and swap it into the next position.
2. Min=1, swap with 8 → 1,4,8,9,3.
3. You still scan the unsorted suffix every pass; there is no early exit like flagged bubble.
4. Classic selection is not stable (equal keys can swap order).

Long

1. Passes put 1, then 3, then 4, then 8,9. Swaps: $8 \leftrightarrow 1$, $4 \leftrightarrow 3$, $8 \leftrightarrow 4$, maybe $8 \leftrightarrow 8$ skip, 9 in place. Bubble does more adjacent swaps. Both $O(n^2)$ comparisons.

Next lecture: 2.4.2 Searching Algorithms: Linear Search